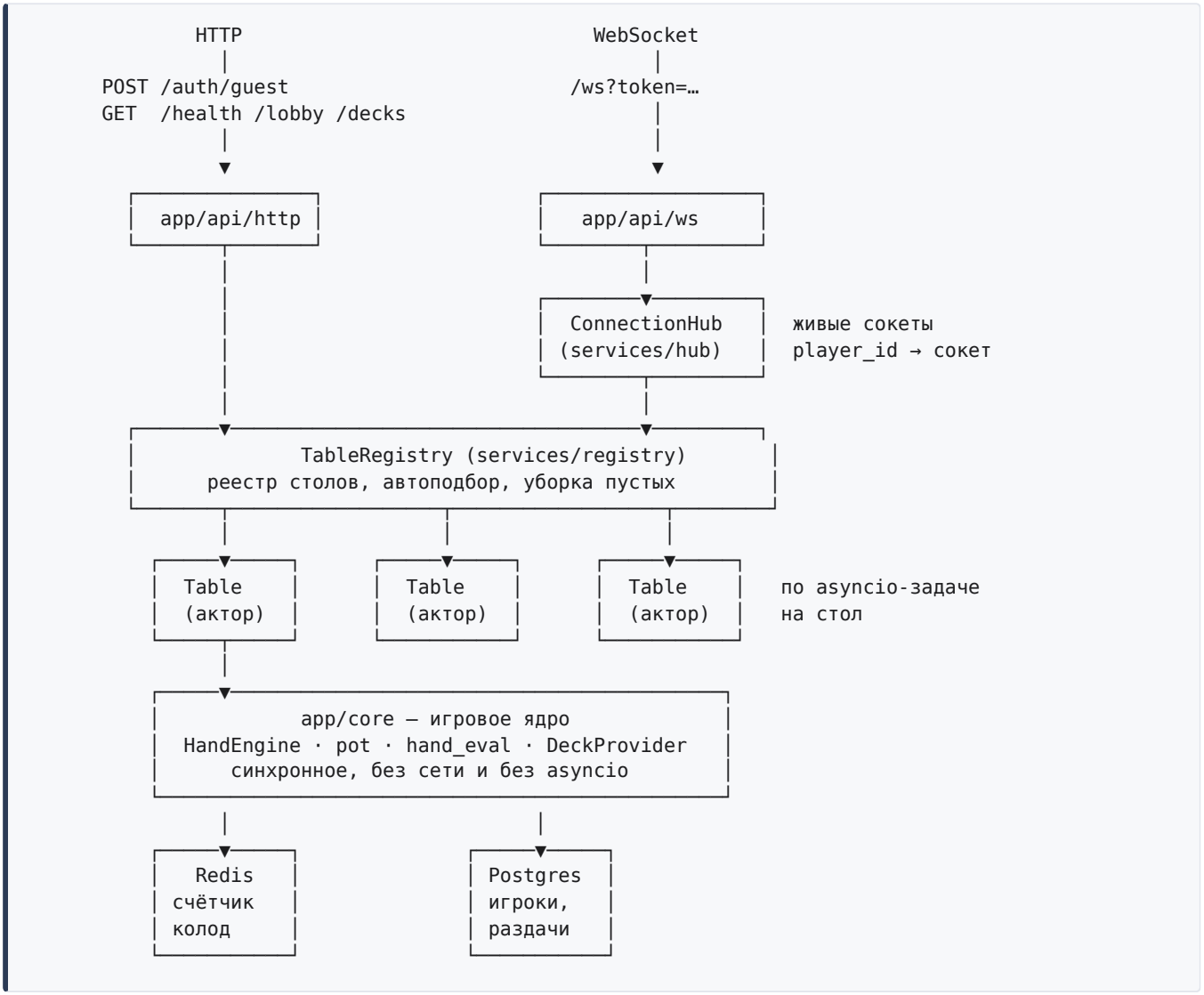
NEON POKER · БЭКЕНД ОНЛАЙН-ПОКЕРА

Архитектура бэкенда

Документ описывает, как устроен сервер, почему он устроен именно так и где находятся точки расширения. Справочник по сообщениям — в PROTOCOL.md, требования и границы этапа — в SPECIFICATION.md.

1. Общая картина



Ключевое разделение проходит по границе `app/core`. Всё, что внутри, — чистые правила покера: синхронный код без сети, без `asyncio` и без обращений к БД. Всё, что снаружи, — доставка: сокеты, таймеры, очереди, хранилища.

Это не эстетика, а практическая вещь. Игровую логику можно прогнать в тестах за микросекунды — марафон в 1900 раздач с проверкой инвариантов на каждой занимает меньше секунды. Если бы правила были размазаны по асинхронному коду, такие тесты

было бы невозможно написать, а найденные в них ошибки (см. раздел 9) уехали бы в прод.

---

2. Стол как актор

Каждый стол — это отдельная `asyncio`-задача со своей очередью команд (`app/services/table.py`). Всё, что меняет состояние стола, попадает в эту очередь и обрабатывается строго по одному:

- игрок сходил;
- истёк таймер хода;
- пора начинать следующую раздачу;
- игрок сел, встал, отключился, вернулся, докупил фишек.

Почему так

Альтернатива — общий объект стола с блокировками. Она плоха тем, что состояние покерного стола меняется из принципиально разных источников (сеть и время), и любая пара «игрок сходил ровно в момент таймаута» превращается в гонку. Живой пример: игрок нажал «колл» в ту же миллисекунду, когда сработал авто-фолд. С блокировками это надо ловить и разруливать; в модели актора это просто две команды в очереди, и вторая увидит уже изменённое состояние и корректно отвалится с `not_your_turn`.

В коде стола нет ни одной блокировки — и это следствие модели, а не дисциплины.

Устройство цикла

```
while не остановлен:
    ждать команду с таймаутом = «сколько до ближайшего дедлайна»
    ├─ пришла команда  → обработать
    └─ таймаут         → обработать дедлайны
    в обоих случаях    → проверить дедлайны ещё раз
```

Последняя строчка важнее, чем кажется. Если проверять дедлайны только по таймауту очереди, то поток команд от клиентов (снапшоты, чат, пинги) не даст очереди опустеть — и таймер хода не сработает никогда. Шести игроков, шлющих сообщения в пределах рейт-лимита, достаточно, чтобы стол перестал раздавать.

Дедлайнов три: время на ход, старт следующей раздачи и периодическая уборка (отвалившиеся игроки, пустые места) раз в 5 секунд.

Живучесть

Задача стола не должна умирать: вместе с ней зависли бы все сидящие за столом клиенты. Поэтому:

- обработка команд и обработка дедлайнов обёрнуты в перехват исключений с логированием — ошибка в одной раздаче не роняет стол;

- если раздача не смогла стартовать, движок сбрасывается, **стеки восстанавливаются из снимка до блайндов**, клиентам уходит свежий снимок. Без явного отката стол остался бы в состоянии «идёт раздача» навсегда — тихий вечный ступор хуже громкой ошибки;
- при остановке стола все ожидающие ответа команды принудительно завершаются ошибкой, иначе обработчик сокета ждал бы ответа вечно;
- у ожидания ответа есть страховочный таймаут (15 с) — клиент получит внятную ошибку, даже если стол залип в неожиданном месте.

---

3. Жизненный цикл раздачи

`HandEngine` (`app/core/game.py`) — синхронная машина состояний. На вход: места, позиция баттона, блайнды, курсор колоды. На выход: список событий, которые стол рассылает клиентам.

```
start()
 ├─ определить SB и BB (в хедз-апе баттон = SB)
 ├─ поставить блайнды (возможен олл-ин прямо на блайнде)
 ├─ раздать по 2 карты, начиная с SB, по одной по кругу
 └─ найти первого, кто МОЖЕТ ходить
      └─ никто не может → сразу открывать улицы

act(place, действие, сумма)
 ├─ проверить легальность по legal_actions()
 ├─ подвинуть фишки, обновить current_bet / min_raise
 ├─ полноценный рейз → круг торговли открывается заново
 ├─ неполный олл-ин   → круг НЕ открывается
 └─ дальше:
      ├─ остался один активный → раздача закончена без вскрытия
      ├─ есть кому ходить      → следующий игрок
      └─ иначе                 → следующая улица

следующая улица
 ├─ сжечь карту, открыть флоп / тёрн / ривер
 ├─ торговаться некому → сразу следующая улица (автопрогон борда)
 └─ после ривера → вскрытие

вскрытие
 ├─ вернуть неколированный излишек ставки
 ├─ оценить руки, разложить банки, распределить
 └─ hand_finished
```

Границы ответственности: движок не знает про время, сокеты и игроков вне раздачи. Таймеры, рассылку и посадку делает стол.

Правила, на которых легко ошибиться

Три места, где реализация специально отличается от «наивной»:

Неполный олл-ин не открывает торговлю заново. Если игрок пошёл олл-ин на сумму меньше полного рейза, те, кто уже ходил, могут только уравнять или сбросить — но не переповысить. Отслеживается множеством «кто ходил после последнего полноценного

рейза». Кнопка «олл-ин» тоже подчиняется этому запрету: иначе она стала бы дырой, через которую ре-рейз проходит в обход.

Повышать некому — значит нельзя. Если единственный оставшийся соперник уже в олл-ине, повышение не предлагается вовсе. Максимум ставки ограничен эффективным стеком соперников, чтобы клиент не рисовал мёртвый слайдер.

Первым ходит тот, кто может ходить. Формальная позиция (баттон в хедз-апе, следующий за BB в остальных случаях) — это только точка отсчёта. Дальше движок ищет первого, кто реально способен действовать. Иначе блайнд, ушедший в олл-ин при постановке, получал бы ход — и по таймауту сервер «сбрасывал» бы карты за игрока, который уже поставил всё.

---

4. Банки и сайд-поты

`app/core/pot.py` — две функции, обе чистые.

`build_pots` раскладывает вклады игроков на слои:

1. Сначала возвращается неколированный излишек: если один вложил больше, чем кто-либо смог уравнять, разница физически не участвует в игре.
2. Затем вклады режутся по уровням. Каждый уровень даёт банк, претенденты — те, кто дошёл до этого уровня и не сбросил карты.
3. Слои с одинаковым составом претендентов схлопываются в один банк.
4. Слой, который сложили **только спасовавшие**, не выбрасывается, а прирастает к ближайшему банку с претендентами. Это редкий случай (нужно, чтобы крупные вкладчики спасовали, а до вскрытия дошёл коротышка), но без такой обработки фишки просто исчезали бы со стола.
5. На выходе проверяется инвариант: разложено ровно столько, сколько вложено. Расхождение — исключение, а не молчаливая потеря.

`distribute` раздаёт каждый банк лучшим рукам среди претендентов. Нечётные фишки при сплите уходят первому слева от баттона, по каждому банку отдельно.

---

5. Оценка комбинаций

`app/core/hand_eval.py` собирает лучшую пятёрку из 5–7 карт аналитически, без перебора и без таблиц предпросчёта. Результат — кортеж `key`, который сравнивается напрямую.

Одна деталь стоит отдельного упоминания, потому что цена ошибки здесь высокая: **сравнение на равенство идёт только по `key`, а не по конкретным картам.** У двух игроков с одинаковым по силе стритом лучшие пятёрки почти всегда состоят из разных карт — каждый играет свою десятку. Если бы равенство учитывало набор карт, сплит-поты

не работали бы вовсе и банк целиком уходил бы одному из двух равных игроков. Проверено: на 200 тысячах вскрытий такое расхождение возникает в 64 % случаев равных рук.

---

6. Колоды

Требование проекта — **никакой генерации случайных чисел во время игры**.

- `data/decks.json` содержит 10 колод. Сгенерированы один раз скриптом `scripts/generate_decks.py`: Fisher–Yates, где источник «случайности» — поток SHA-256 от фиксированного сида. Файл воспроизводим байт в байт, что проверяется тестом.
- `DeckProvider` в рантайме только читает файл и раздаёт колоды по кругу: `INCR` одного глобального ключа в Redis → `deck_id = (номер − 1) mod 10`. Счётчик общий на все столы и все инстансы сервера, поэтому две раздачи, начавшиеся одна за другой, гарантированно получат разные колоды.
- При недоступном Redis провайдер переходит на локальный счётчик процесса и пишет об этом в лог: игра продолжается, но уникальность между инстансами уже не гарантируется.
- Каждая раздача несёт `deck_id` и `deck_sequence` — по ним раздачу можно воспроизвести и проверить постфактум.
- `GET /decks` отдаёт контрольные суммы колод и позицию счётчика. Сами карты наружу не отдаются: иначе, зная позицию, клиент вычислил бы будущие раздачи.

Увеличить число вариантов: `python scripts/generate_decks.py --count 100`. Формат файла и код не меняются.

Ограничение, которое надо понимать

Фиксированный набор — сознательный выбор ради воспроизводимости, но он же означает, что игрок, увидевший 10 раздач, знает все возможные колоды, а зная позицию счётчика, предскажет следующую. Для игры на реальные деньги этого недостаточно: нужен криптостойкий шафл на раздачу плюс commit-reveal — сервер публикует хеш колоды до раздачи и саму колоду после.

Точка расширения одна: `DeckProvider.next_deck()`. Это единственное место, где решается, какая колода пойдёт в игру, и подменяется оно без изменений в движке.

---

7. Соединения, посадка, реконнект

`ConnectionHub` держит соответствие `player_id → сокет`. Одному игроку — одно активное соединение; второй коннект вытесняет первый с кодом `4001`. Место за столом при этом не теряется: для стола игрок никуда не уходил.

`TableRegistry` — реестр столов и автоподбор. Стратегия подбора: среди столов нужного уровня ставок со свободным местом берётся **самый заполненный**. Так столы доукомплектовываются, а не размазывают игроков по десятку полупустых — за 6-max это прямо влияет на качество игры.

Две тонкости, обе про гонки:

- Пока идёт посадка, у стола поднят счётчик `pending_joins`, и уборщик пустых столов такой стол не трогает. Иначе стол могли снести между выбором и посадкой — с уже списанным бай-ином, который некуда вернуть.
- На время подбора игрок помечается в реестре как «занят». Без этого два одновременных `quick_join` одного игрока (вторая вкладка, обрыв на мобильной сети) сажали его за два разных стола и списывали бай-ин дважды.

Реконнект. Место держится `RECONNECT_WINDOW_SECONDS` (по умолчанию 120 с). Раздача продолжается: подошла очередь, а игрока нет — авто-фолд, после трёх подряд игрока переводят в сит-аут. При возврате клиент получает полный снимок стола со своими картами, бордом, банком и остатком времени на ход — этого достаточно, чтобы отрисовать всё с нуля.

Если игрок не вернулся за отведённое окно, место освобождается, стек возвращается на баланс.

---

8. Хранилища

PostgreSQL — игроки и балансы; опционально история раздач (флаг `PERSIST_HAND_HISTORY`, по умолчанию выключен). Схема в `app/db/models.py`, доступ через репозитории.

Redis — глобальный счётчик колод. В перспективе сюда же лягут регистр «стол → инстанс» и pub/sub между инстансами.

Оба хранилища необязательны для того, чтобы играть: при их отсутствии сервер пишет предупреждение и продолжает работать на памяти. Это осознанное решение — поднять и посмотреть игру должно быть можно одной командой, без инфраструктуры. В проде обе зависимости, разумеется, обязательны.

Балансы кэшируются в памяти (`AccountService`) и досылаются в БД: ходить в Postgres на каждое движение фишек незачем.

---

9. Тесты

`python tests/run_all.py` или `pytest` — 60 тестов, все на игровое ядро.

Файл	Что проверяет
`test_hand_eval`	категории комбинаций, колесо A-2-3-4-5, кикеры, равенство рук
`test_pot`	сайд-поты, возврат излишка, нечётные фишки, «мёртвые» слои
`test_deck_provider`	валидность файла колод, воспроизводимость, выдача по кругу
`test_game_flow`	очередь хода, опция BB, мин-рейз, олл-ины, сплиты
`test_simulation`	~1900 раздач ботами на 2/3/6 игроков, включая микро-стеки

Марафон проверяет на **каждой** раздаче: сумма фишек не изменилась, карты не дублируются, стеки не ушли в минус, ни одно место не зависло в промежуточном состоянии, раздача завершилась.

Именно инварианты, а не ожидаемые значения, ловят интересные ошибки. Из найденных при разработке: исчезающие фишки в сайд-поте, где все вкладчики спасовали (117 раздач из 30 000, суммарно 6 075 фишек), и неработающие сплит-поты. Ни то, ни другое не нашлось бы точечными тестами — нужно было именно много раздач с проверкой сохранения фишек.

---

10. Что дальше

Порядок по важности:

1. **Настоящая авторизация.** Сейчас гостевой JWT. Интерфейс узкий — `issue_guest_token` / `decode_token` / `TokenClaims` в `app/auth/tokens.py`; всё остальное менять не придётся.
2. **Горизонтальное масштабирование.** Стол живёт в памяти процесса. Redis уже подключён; нужен регистр `table_id → инстанс` и роутинг сокета на нужный инстанс, либо sticky-сессии на балансировщике.
3. **Рейк.** Точка вставки одна: `HandEngine._finish`, перед распределением банков.
4. **Честность раздачи** — криптостойкий шафл и commit-reveal, см. раздел 6.
5. **Наблюдаемость и античит.** История раздач уже пишется по флагу; дальше — метрики и детект сговора.
6. **Турниры.** Растущие блайнды, выбывание, балансировка столов. Ядро к этому готово: `HandEngine` ничего не знает про формат стола.